Solution: Middle of the LinkedList

Problem Statement

Given the head of a **Singly LinkedList**, write a method to return the **middle node** of the LinkedList.

If the total number of nodes in the LinkedList is even, return the second middle node.

Example 1:

```
Input: 1 -> 2 -> 3 -> 4 -> 5 -> null
Output: 3
```

Example 2:

```
Input: 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> null
Output: 4
```

Example 3:

```
Input: 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> 7 -> null
Output: 4
```

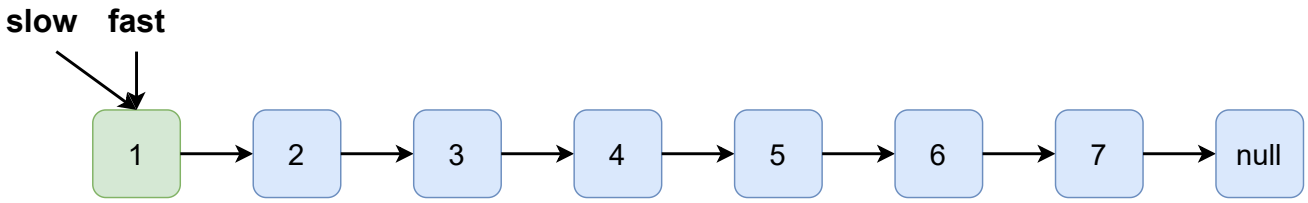
Solution

One brute force strategy could be to first count the number of nodes in the LinkedList and then find the middle node in the second iteration. Can we do this in one iteration?

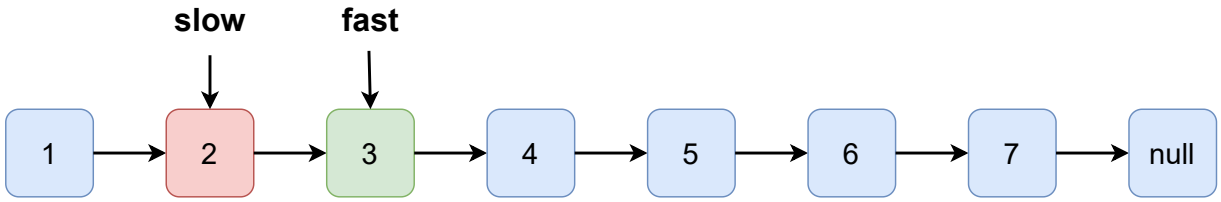
We can use the **Fast** & **Slow pointers** method such that the fast pointer is always twice the nodes ahead of the slow pointer. This way, when the fast pointer reaches the end of the LinkedList, the slow pointer will be pointing at the middle node.

let's visualize example 3 via the below diagram.

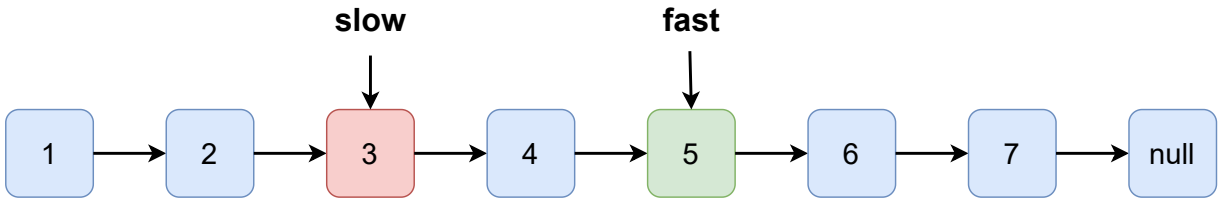
Using fast and slow pointers to find the middle of the linked list.

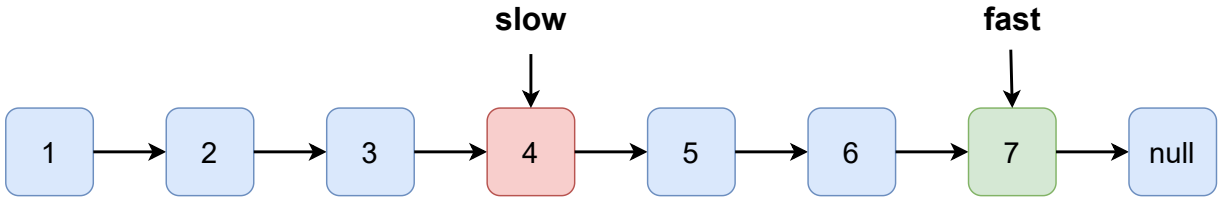


Move slow pointer one step, and fast pointer two step in each iteration.



Again, move slow pointer one step, and fast pointer two step in each iteration.





fast.next = null. So, slow.val is middle of the linked list.

Code

Here is what our algorithm will look like:

Java

```
/*class ListNode {
  int val = 0;
  ListNode next;

  ListNode(int val) {
    this.val = val;
  }
}*/

public class Solution {
```

```
12      public ListNode findMiddle(ListNode head) {
13        ListNode slow = head;         // Initialize slow pointer at the head of the list
14        ListNode fast = head;         // Initialize fast pointer at the head of the list
15        while (fast != null && fast.next != null) {  // Traverse the list using slow and fast pointers
16          slow = slow.next;          // Move slow pointer one step at a time
17          fast = fast.next.next;     // Move fast pointer two steps at a time
18        }
19
20        return slow;  // Return the node pointed to by slow (middle node or middle-left node)
21      }
22
23      public static void main(String[] args) {
```

Run Code Result | Stdout

You must run your code first

Execute

Time Complexity

The above algorithm will have a time complexity of O(N) where 'N' is the number of nodes in the LinkedList.

Space Complexity

The algorithm runs in constant space O(1).

← Previous

Middle Of The LinkedList (Easy)

Next →

Start Of LinkedList Cycle (Medium)

Mark as Completed